Neovim Cheat Sheet

Modern Python/Rust setup - leader key is Space

Most useful everyday shortcuts

Key	Action	Where / notes
<leader>ff	Find files	Telescope
<leader>fg	Find text in project	Telescope live grep
<leader>e	Toggle file explorer	Neo-tree
<S-h> / <S-l>	Previous / next buffer	Bufferline
<leader>w	Save file	Core
<Esc>	Clear search highlight	Core
K	Hover documentation	LSP
gd / gr	Definition / references	LSP
<leader>rn	Rename symbol	LSP
<leader>ca	Code action	LSP
[d /]d	Previous / next diagnostic	LSP
<leader>cf	Format file	Conform
<leader>tt	Run nearest test	Neotest
<leader>tf	Run current test file	Neotest
<leader>db	Toggle breakpoint	DAP
<leader>dc	Start / continue debugger	DAP
<leader>-	Open Yazi at current file	Yazi
<leader>ua	Toggle ANSI colors	ansi.nvim

*Tip: **<leader>** means **Space**. Uppercase key names mean Shift, for example **<leader>tO** means Space, t, Shift-o. Press Space and pause to let which-key show available mappings.*

All bindings by area

Core editor, navigation, buffers, git, LSP, formatting, tests, debugging, Rust, Yazi, ANSI logs, and VS Code mode.

Core editor

Key	Action
`<Esc>`	Clear search highlight
`<leader>w`	Save file
`<leader>q`	Quit window
`<leader>h`	Horizontal split
`<leader>v`	Vertical split
`<C-h>`	Move to left window
`<C-j>`	Move to lower window
`<C-k>`	Move to upper window
`<C-l>`	Move to right window
`j / k`	Move by visual lines
`< / >`	Indent selection and keep it selected

Buffers and UI tabs

Key	Action
`<leader>bn`	Next buffer
`<leader>bN`	Previous buffer; capital N
`<leader>bd`	Delete buffer
`<S-h>`	Previous buffer tab
`<S-l>`	Next buffer tab
`<leader>bp`	Pin buffer
`<leader>bP`	Close unpinned buffers; capital P
`<leader>bo`	Close other buffers
`<leader>br`	Close buffers to the right
`<leader>bl`	Close buffers to the left

Find files / project navigation

Key	Action
`<leader>ff`	Find files
`<leader>fg`	Find text
`<leader>fb`	Find open buffers
`<leader>fh`	Find help tags
`<leader>fr`	Recent files
`<leader>e`	Toggle file explorer

Formatting and completion

Key	Action
`<leader>cf`	Format current file
`<C-Space>`	Show completion menu
`<C-e>`	Hide completion menu
`<CR>`	Accept selected completion
`<Tab>`	Next item / snippet jump
`<S-Tab>`	Previous item / snippet jump back
`:ConformInfo`	Show formatter status

LSP and diagnostics

Key	Action	Where / notes
`K`	Hover documentation	Any attached LSP
`gd`	Go to definition	Any attached LSP
`gD`	Go to declaration	Any attached LSP
`gi`	Go to implementation	Any attached LSP
`gr`	Go to references	Any attached LSP
`<leader>rn`	Rename symbol	Any attached LSP
`<leader>ca`	Code action	Any attached LSP
`[d`	Previous diagnostic	Shows diagnostic float
`]d`	Next diagnostic	Shows diagnostic float
`<leader>de`	Show diagnostic at cursor	Diagnostic float
`<leader>dq`	Diagnostics to quickfix	Quickfix list

Git / Gitsigns / Fugitive

Key	Action	Where / notes
`]h`	Next git hunk	Gitsigns
`[h`	Previous git hunk	Gitsigns
`<leader>hs`	Stage hunk	Normal and visual mode
`<leader>hr`	Reset hunk	Normal and visual mode
`<leader>hp`	Preview hunk	Gitsigns
`<leader>hb`	Blame current line	Gitsigns
`:Git`	Open Fugitive git command	Fugitive command, no direct key

Testing - pytest via Neotest

Key	Action	Where / notes
<leader>tt	Run nearest test	Neotest
<leader>tf	Run current test file	Neotest
<leader>td	Debug nearest test	Neotest using DAP
<leader>ts	Toggle test summary	Neotest
<leader>to	Open test output	Focused output window
<leader>tO	Toggle test output panel	Persistent panel; capital O
<leader>tw	Toggle watch current file	Neotest watch
<leader>tS	Stop test run	Neotest; capital S

Debugging - nvim-dap / debugpy

Key	Action	Where / notes
<leader>db	Toggle breakpoint	nvim-dap
<leader>dB	Set conditional breakpoint	Prompts for condition; capital B
<leader>dc	Start / continue debugger	nvim-dap
<leader>do	Step over	nvim-dap
<leader>di	Step into	nvim-dap
<leader>dO	Step out	nvim-dap; capital O
<leader>dr	Open debug REPL	nvim-dap
<leader>du	Toggle debug UI	nvim-dap-ui
<leader>dx	Terminate debug session	nvim-dap

Rust - rustaceanvim

Key	Action
<leader>rh	Rust hover actions
<leader>ra	Rust code action
<leader>rr	Rust runnables
<leader>re	Explain Rust error
<leader>rd	Render Rust diagnostic
<leader>rc	Open Cargo.toml
<leader>ro	Open Rust docs

Yazi and ANSI logs

Key	Action
<leader>-	Open Yazi at current file
<leader>cw	Open Yazi in current working directory
<C-Up>	Resume / toggle Yazi session
<leader>ua	Toggle ANSI colors
:AnsiEnable	Enable ANSI rendering
:AnsiDisable	Disable ANSI rendering
:AnsiToggle	Toggle ANSI rendering

VS Code Neovim mode

Key	Action	Where / notes
<leader>w	Save file	VS Code action
<leader>q	Close active editor	VS Code action
<leader>ff	Quick open file	VS Code action
<leader>fg	Find in files	VS Code action
<leader>e	Open Explorer view	VS Code action
<leader>p	Command palette	VS Code action
<leader>rn	Rename symbol	VS Code action
<leader>ca	Quick fix / code action	VS Code action
gd	Go to definition	VS Code action
gr	Go to references	VS Code action

Useful commands

Key	Action	Where / notes
`:Lazy sync`	Install/update plugins	Plugin manager
`:Mason`	Open Mason tool manager	External tools
`:checkhealth vim.lsp`	Inspect active LSP clients	Modern replacement for LspInfo style checks
`:checkhealth vim.deprecated`	Check deprecation warnings	Useful after plugin updates
`:ConformInfo`	Inspect formatter availability	Conform
`:checkhealth nvim-treesitter`	Check Treesitter	Parser health

Plugin ownership mental model

Key	Action
`core/`	Basic editor behavior: options, keymaps, autocmds, LSP keymaps
`config/`	lazy.nvim bootstrap and plugin loading
`plugins/lsp.lua`	General LSP servers: Lua, Python, Bash, JSON, YAML
`plugins/rust.lua`	Rust-specific integration through rustaceanvim
`plugins/formatting.lua`	Conform formatting rules
`plugins/tools.lua`	Mason-installed non-LSP tools like stylua, shfmt, prettier, debugpy
`plugins/testing.lua`	Neotest and pytest integration
`plugins/debugging.lua`	DAP, debug UI, virtual text, Python debugpy adapter
`plugins/ui.lua`	Lualine statusline and Bufferline tabs
`plugins/yazi.lua`	Yazi terminal file manager integration
`plugins/ansi.lua`	ANSI color rendering for log files
`vscode_config/`	VS Code Neovim-only mappings